

Diego Chuquillanqui Cuenca

During the final part of the Sensing the World module, my group and I applied the concepts from the previous weeks to design and build an automatic floodgate prototype. We began by completing the project planning worksheet, where we defined the problem we wanted to address, the purpose of the system, its inputs and outputs, and the main requirements for the prototype. Our goal was to create a small-scale flood-protection system that could detect unsafe moisture conditions and automatically close a gate to help protect people, homes, or belongings from flooding. Our initial design included an Arduino Nano ESP32, our homemade moisture sensor, LEDs, a push button, and a servo motor to move a gate made from materials such as cardboard and popsicle sticks. We also planned for the green LED to represent safe conditions, the red LED to represent unsafe conditions, and the push button to provide manual control over the system.

Once we began physically building the prototype, several parts of the original design changed. We decided to use both the PEDOT sensor that I had fabricated earlier in the module and a commercial rain sensor. The PEDOT sensor allowed us to detect changes in humidity or moisture, while the rain sensor gave us a more direct indication of water contact. We also found that a servo motor was not the best option for the movement we wanted, so we changed to a stepper motor. We connected the stepper to a driver and created a pulley and string mechanism that could raise and lower the gate. At the same time, we assembled the circuit on a breadboard with the Arduino, sensors, red and green LEDs, reset button, and motor driver. We first tested many of these components separately before integrating them into the complete system. This included test programs for the stepper motor, rain sensor, and PEDOT sensor so that we could understand how each component behaved on its own.

Programming and troubleshooting became a major part of the project as we integrated everything together. We used AI to generate and revise the MicroPython code for the full system and to create smaller programs for testing individual components. We also used it to troubleshoot problems that appeared during construction. For example, it helped us determine that the stepper motor was better suited to the gate mechanism than the original servo, and it helped us work through wiring, grounding, and motor-driver problems when the motor was not operating correctly. We learned that giving AI detailed information was important, so I kept relevant course materials, previous code, pin assignments, hardware information, and our observations together as context. We still reviewed the suggestions ourselves and physically tested them before deciding whether they should become part of the design.

Testing the complete system also showed us that real sensors do not always behave as simply as expected. We had to establish thresholds for the two sensors and repeatedly test how the gate responded. One challenge with the PEDOT sensor was that after being exposed to humidity, its reading could take time to return toward its original dry value. This meant we had to account for sensor recovery when developing the control logic and determining when conditions could be considered safe again. In the final system, the gate begins open with the green LED on. If either sensor detects an unsafe condition, the red LED turns on and the stepper motor closes the gate. Reopening requires a manual reset, and the gate can only reopen once both

sensors indicate safe conditions. We chose this design so the system would not depend completely on automation and accidentally reopen while conditions might still be unsafe.

After completing and testing the prototype, we demonstrated the system and then prepared our final presentation. For the presentation, we divided the project into its main stages, including sensing, processing, actuation, and reset, and explained how AI supported the development process. Preparing the presentation made me think about the project differently because we had to explain not only what the floodgate did, but also why we made specific design choices. The final system still met our original goal of detecting unsafe flood conditions and closing the gate, but the way we achieved that goal changed as we tested and improved the prototype.

Overall, this project brought together many of the concepts I had worked with throughout the Sensing the World module. I was able to use the PEDOT sensor I had fabricated earlier and combine it with what I had learned about microcontrollers, circuits, sensors, motors, and MicroPython to create one complete system. The most important thing I learned was that engineering design is iterative. A design that looks reasonable during planning can still require changes once it is physically constructed and tested. Changing the motor, troubleshooting the electronics, adjusting the sensor logic, and adding the manual safety reset all came from observing how the real prototype behaved. I also learned that AI can be useful for programming and troubleshooting when it is given enough context, but its suggestions still need to be understood and tested. By using AI for some of the coding work, I was able to spend more time on sensor integration, physical prototyping, testing, and improving the design, which were some of the most valuable parts of the experience.